

Публичное акционерное общество «Ростелеком»
(ПАО «Ростелеком»)

**ОПИСАНИЕ ДОРАБОТАННОГО ПРОТОКОЛА
ТАЙНОГО ГОЛОСОВАНИЯ**

Листов 25

Москва 2023

СОДЕРЖАНИЕ

1	Введение	4
2	Базовые криптографические механизмы	5
2.1	Хэш-функция.....	6
2.2	Схема подписи	6
2.3	Схема подписи вслепую	7
2.4	Специальная схема шифрования (с открытым ключом).....	9
2.5	Схема разделения секрета (с доверенным дилером)	13
2.6	Схема коммитмента (обязательств)	14
3	Участники протокола.....	16
4	Этапы протокола	17
4.1	Формирование исходных данных	17
4.1.1	Формирование списков избирателей	17
4.1.2	Передача данных голосования.....	17
4.2	Инициализация системы.....	17
4.2.1	Генерация ключей.....	17
4.2.2	Загрузка данных голосования	17
4.2.3	Загрузка ключа шифрования бюллетеней	18
4.3	Предоставление права получения бюллетеня	18
4.3.1	Идентификация и аутентификация избирателя.....	18
4.3.2	Получение подписи вслепую и ключа шифрования бюллетеней	18
4.4	Подача заполненного бюллетеня	19
4.4.1	Волеизъявление.....	19
4.4.2	Публикация бюллетеня	20
4.4.3	Проверка бюллетеня.....	20
4.5	Подведение итогов	20
4.5.1	Прекращение выдачи и приема бюллетеней.....	20
4.5.2	Восстановление разделяемого ключа шифрования	21
4.5.3	Подсчет итогов.....	21
4.5.4	Расшифрование итогов	21

4.6	Аудит	21
4.7	Публикация результатов.....	22
5	Совместимость с предыдущими версиями протокола.....	22
5.1	Возможность изменения протокола подписи вслепую	22
5.1.1	Схема подписи вслепую на основе RSA.....	22
5.1.2	Описание конструкции FDH	23
	Список литературы.....	25

1 Введение

В настоящем документе приведено описание порядка взаимодействия логических элементов системы дистанционного электронного голосования, называемого далее протоколом ДЭГ. В разделе 2 описаны криптографические механизмы, которые используются в протоколе, в разделе 3 перечислены логические участники взаимодействия, в разделе 4 описаны шаги выполнения протокола, формирующие порядок взаимодействия участников.

2 Базовые криптографические механизмы

В настоящем разделе описаны криптографические механизмы, используемые в протоколе ДЭГ.

Обозначим:

- H – хэш-функция Стрибог с длиной выхода 256 битов, определенная в стандарте ГОСТ Р 34.11–2012 [2];
- $HMAC$ – функция $HMAC_GOSTR3411_2012_256$ с длиной ключа 256 бит, определенная в Рекомендациях по стандартизации Р 50.1.113–2016 [3];
- $\parallel$ – конкатенация двоичных строк;
- $a[i; j] = (a_i, \dots, a_j)$ – подстрока байтовой строки $a = (a_0, \dots, a_{t-1})$;
- $str_t(r)$ – функция, преобразующая целое число $r = 256^{t-1}b_{t-1} + 256^{t-2}b_{t-2} + \dots + 256b_1 + b_0$, $b_i \in \{0, \dots, 255\}$, $i = 0, \dots, t-1$, в байтовую строку $b_0 \parallel \dots \parallel b_{t-1}$;
- $STR_t(r)$ – функция, преобразующая целое число $r = 256^{t-1}b_{t-1} + 256^{t-2}b_{t-2} + \dots + 256b_1 + b_0$, $b_i \in \{0, \dots, 255\}$, $i = 0, \dots, t-1$, в байтовую строку $b_{t-1} \parallel \dots \parallel b_0$;
- $int(b)$ – функция, преобразующая байтовую строку $b = b_0 \parallel \dots \parallel b_{t-1}$ в целое число $r = 256^{t-1}b_{t-1} + 256^{t-2}b_{t-2} + \dots + 256b_1 + b_0$;
- $x \stackrel{u}{\leftarrow} X$ – элемент x выбирается из множества X случайно в соответствии с равномерным распределением;
- $\{0, 1\}^n$ – множество всех n -битовых строк, где n – натуральное число;
- $\{0, 1\}^*$ – множество всех битовых строк конечной длины, включая пустую строку;
- $\perp$ – символ, обозначающий событие "ошибка";
- $x \bmod y$ – остаток от деления целого числа x на целое число y ;
- $\oplus$ – операция покомпонентного сложения по модулю 2 двух битовых строк одинаковой длины;
- $\mathcal{E}$ – эллиптическая кривая $id\text{-}tc26\text{-}gost\text{-}3410\text{-}2012\text{-}256\text{-}paramSetB$, параметры которой определены в Рекомендациях по стандартизации Р 1323565.1.024–2019 [4];
- q – простое число, порядок группы точек эллиптической кривой $\mathcal{E}$;
- P – образующий элемент группы точек эллиптической кривой $\mathcal{E}$;
- $\mathcal{O}$ – нейтральный элемент группы точек эллиптической кривой $\mathcal{E}$;
- $\mathbb{Z}_q$ – конечное поле порядка q , где q – простое;
- $\mathbb{Z}_q^*$ – мультипликативная группа конечного поля $\mathbb{Z}_q$.

Всюду далее открытые ключи (ключи шифрования, ключ проверки подписи) являются элементами группы точек эллиптической кривой $\mathcal{E}$, закрытые ключи (ключи расшифрования, ключ подписи) являются элементами мультипликативной группы конечного поля $\mathbb{Z}_q^*$.

Все элементы конечного поля $\mathbb{Z}_q$, передаваемые и используемые в рамках протоколов, описанных в настоящем документе, представляются в виде байтовых строк в формате little-endian длины 32 байта. Все точки с координатами (x, y) эллиптической кривой, передаваемые и используемые в рамках протоколов, описанных в настоящем документе, представляются в виде байтовой строки $b_x \parallel b_y$, где b_x и b_y — байтовое представление x -координаты и y -координаты точки соответственно в формате little-endian, длины 32 байта.

2.1 Хэш-функция

Хэш-функция, отображающая двоичные строки конечной длины в элементы конечного поля $\mathbb{Z}_q$, определяется функцией Hash.

Вход:

- сообщение $m \in \{0,1\}^*$,
- идентификатор области применения DST длины l байт.

Выход:

- хэш-значение $h \in \mathbb{Z}_q$.

```
Hash(m, DST)


---


 $L \leftarrow \left\lceil \frac{\lceil \log_2(q) \rceil + 128}{8} \right\rceil$ 
 $L' \leftarrow \lceil L/32 \rceil$ 
 $DST' \leftarrow DST \parallel \text{STR}_1(l)$ 
 $Z_{\text{pad}} \leftarrow \text{STR}_{64}(0)$ 
 $m' \leftarrow Z_{\text{pad}} \parallel m \parallel \text{STR}_2(L) \parallel \text{STR}_1(0) \parallel DST'$ 
 $w_0 \leftarrow H(m')$ 
 $w_1 \leftarrow H(w_0 \parallel \text{STR}_1(1) \parallel DST')$ 
 $w_i \leftarrow H(w_0 \oplus w_{i-1} \parallel \text{STR}_1(i) \parallel DST'), \quad i = 2, \dots, L'$ 
 $w \leftarrow w_1 \parallel \dots \parallel w_{L'}$ 
 $W \leftarrow w[0; L - 1]$ 
 $h \leftarrow \text{int}(W) \bmod q$ 
return  $h$ 
```

2.2 Схема подписи

Схема подписи S определяется следующими процессами (используется схема, определенная стандартом ГОСТ Р 34.10-2012 [1]).

Генерация ключей S . KeyGen

Выход:

- ключ подписи sk ,
- ключ проверки подписи pk .

Формирование подписи S . Sign

Вход:

- ключ подписи sk ,
- сообщение m .

Выход:

- подпись σ .

Проверка подписи S . Verify

Вход:

- ключ проверки подписи pk ,
- сообщение m ,
- подпись σ .

Выход:

- бит $b \in \{0,1\}$: $b = 1$ ($b = 0$) – проверка завершилась (не)успешно.

2.3 Схема подписи вслепую

Обозначим: $Hash_{blind}(m) = Hash(m, DST_{blind})$, где

$DST_{blind} = \langle \text{BlindSign-TeZhu-V00-H2F:id-tc26-gost-3410-2012-256-paramSetB_Streebog-256_XMD_RO} \rangle$.

Схема подписи вслепую BS определяется следующими процессами (используется схема, определенная в работе [9]).

Генерация ключей $BS.KeyGen$

Выход:

- ключ подписи sk ,
- ключ проверки подписи pk .

```
KeyGen( )
-----
 $d, z \xleftarrow{U} \mathbb{Z}_q^*$ 
 $Q \leftarrow dP$ 
 $Z \leftarrow zP$ 
 $(sk, pk) \leftarrow (d, (Q, Z))$ 
return  $(sk, pk)$ 
```

Формирование подписи вслепую $\langle BS.Sign(sk, pk), BS.User(pk, m) \rangle$

Вход функции $Sign$:

- ключ подписи sk ,
- ключ проверки подписи pk .

Вход функции $User$:

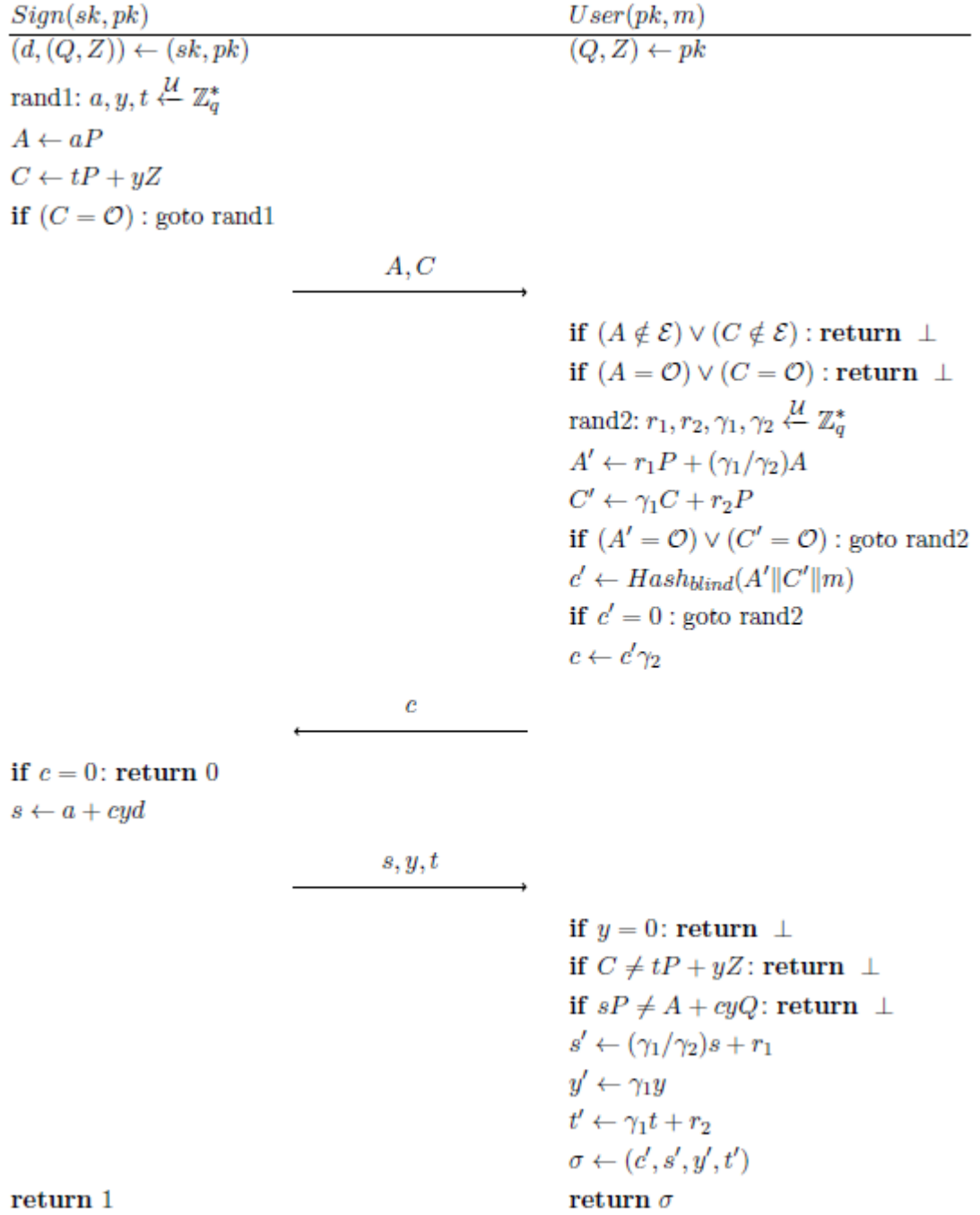
- ключ проверки подписи pk ,
- сообщение m .

Выход функции $Sign$:

- бит $b \in \{0,1\}$: $b = 1$ ($b = 0$) – выполнение протокола завершилось (не)успешно.

Выход функции $User$:

- значение подписи σ .



Проверка подписи $BS.Verify$

Вход:

- ключ проверки подписи pk ,
- сообщение m ,
- значение подписи σ .

Выход:

- бит $b \in \{0,1\}$: $b = 1$ ($b = 0$) – проверка завершилась (не)успешно.

```

Verify( $pk, m, \sigma$ )
( $Q, Z$ )  $\leftarrow pk$ 
( $c, s, y, t$ )  $\leftarrow \sigma$ 
if ( $y = 0$ )  $\vee$  ( $c = 0$ ): return 0
 $C \leftarrow tP + yZ$ 
 $A \leftarrow sP - cyQ$ 
if ( $A = \mathcal{O}$ )  $\vee$  ( $C = \mathcal{O}$ ): return 0
if  $c = \text{Hash}_{\text{blind}}(A \| C \| m)$ : return 1
else : return 0

```

2.4 Специальная схема шифрования (с открытым ключом)

Обозначим: $\text{Hash}_{DP}(m) = \text{Hash}(m, \text{DST}_{DP})$, где:

$\text{DST}_{DP} = \text{«ZKP-CP-eqdllog-V00-H2F:id-tc26-gost-3410-2012-256-paramSetB_Streebog-256_XMD_RO»}$.

Специальная схема шифрования SE построена на основе схемы шифрования Эль-Гамала [6] и схем доказательства с нулевым разглашением, определенных в работах [7], [8], и определяется следующими процессами.

Генерация ключа $SE.KeyGen$

Выход:

- частичный ключ расшифрования sk^{part} ,
- частичный ключ шифрования pk^{part} .

```

KeyGen( )
 $d \xleftarrow{\mathcal{U}} \mathbb{Z}_q^*$ 
 $Q \leftarrow dP$ 
 $(sk^{part}, pk^{part}) \leftarrow (d, Q)$ 
return  $(sk^{part}, pk^{part})$ 

```

Агрегирование ключа $SE.KeyAgg$

Вход:

- частичный ключ шифрования pk_1^{part} ,
- частичный ключ шифрования pk_2^{part} .

Выход:

- ключ шифрования pk .

```

KeyAgg( $pk_1^{part}, pk_2^{part}$ )
 $(Q_1, Q_2) \leftarrow (pk_1^{part}, pk_2^{part})$ 
 $h_1 \leftarrow H(Q_2 \| Q_1)$ 
 $h_2 \leftarrow H(Q_1 \| Q_2)$ 
 $Q \leftarrow h_1 Q_1 + h_2 Q_2$ 
 $pk \leftarrow Q$ 
return  $pk$ 

```

Шифрование $SE.Enc$

Вход:

- ключ шифрования pk ,
- открытый текст $(m_1, \dots, m_t)$, $t \in \mathbb{N}$,
- параметры min, max , $1 \leq min \leq max < t$.

Выход:

- шифртекст $(c_1, \dots, c_t)$,
- доказательство d корректности шифрования (выполнены условия: $m_i \in \{0,1\}$, $i = 1, \dots, t$, и $min \leq m_1 + \dots + m_t \leq max$).

$Enc(pk, (m_1, \dots, m_t), min, max)$

$Q \leftarrow pk$

for $i = 1, \dots, t$:

$M_i \leftarrow m_i P$

$r_i \xleftarrow{\mathcal{U}} \mathbb{Z}_q^*$

$R_i \leftarrow r_i P$

$C_i \leftarrow M_i + r_i Q$

zkp1: $d_i \leftarrow Rprove(Q, 0, 1, m_i, r_i, R_i, C_i)$

if $d_i = \perp$: **goto** zkp1

$m \leftarrow \sum_{i=1}^t m_i, r \leftarrow \sum_{i=1}^t r_i$

$R \leftarrow \sum_{i=1}^t R_i, C \leftarrow \sum_{i=1}^t C_i$

zkp2: $d_\Sigma \leftarrow Rprove(Q, min, max, m, r, R, C)$

if $d_\Sigma = \perp$: **goto** zkp2

$(c_1, \dots, c_t) \leftarrow (R_1, C_1), \dots, (R_t, C_t)$

$d \leftarrow (d_1, \dots, d_t, d_\Sigma)$

return $((c_1, \dots, c_t), d)$

$Rprove(Q, min, max, m, r, R, C)$

$sum \leftarrow 0$

$i \leftarrow min$

while $i \leq max$:

if $m \neq i$:

$c_i \xleftarrow{\mathcal{U}} \mathbb{Z}_q^*$

$r_i \xleftarrow{\mathcal{U}} \mathbb{Z}_q^*$

$A_i \leftarrow r_i P - c_i R$

$B_i \leftarrow r_i Q - c_i (C - iP)$

$sum \leftarrow sum + c_i \mod q$

else :

$index \leftarrow i$

$tmp \xleftarrow{\mathcal{U}} \mathbb{Z}_q^*$

$A_i \leftarrow tmp \cdot P$

$B_i \leftarrow tmp \cdot Q$

$i \leftarrow i + 1$

$Astr \leftarrow A_{min} \parallel \dots \parallel A_{max}$

$Bstr \leftarrow B_{min} \parallel \dots \parallel B_{max}$

$c \leftarrow H(P \parallel Q \parallel R \parallel C \parallel Astr \parallel Bstr)$

$c_{index} \leftarrow c - sum \mod q$

$r_{index} \leftarrow tmp + c_{index} \cdot r \mod q$

if $(c_{index} = 0) \vee (r_{index} = 0)$:

return $\perp$

$cstr \leftarrow c_{min} \parallel \dots \parallel c_{max}$

$rstr \leftarrow r_{min} \parallel \dots \parallel r_{max}$

$\pi \leftarrow (Astr, Bstr, cstr, rstr)$

return π

Проверка корректности шифрования $SE.VerifyEnc$

Вход:

- ключ шифрования pk ,
- шифртекст $(c_1, \dots, c_t)$,

- доказательство d корректности шифрования,
- параметры $min, max, 1 \leq min \leq max < t$.

Выход:

- бит $b \in \{0,1\}$: $b = 1$ ($b = 0$) – проверка завершилась (не)успешно.

$VerifyEnc(pk, (c_1, \dots, c_t), d, min, max)$

```

 $Q \leftarrow pk$ 
 $(R_1, C_1), \dots, (R_t, C_t) \leftarrow (c_1, \dots, c_t)$ 
 $(d_1, \dots, d_t, d_\Sigma) \leftarrow d$ 
for  $i = 1, \dots, t$  :
     $b_i \leftarrow Rverify(Q, 0, 1, R_i, C_i, d_i)$ 
    if  $b_i = 0$  :
        return 0
 $R \leftarrow \sum_{i=1}^t R_i, C \leftarrow \sum_{i=1}^t C_i$ 
 $b_\Sigma \leftarrow Rverify(Q, min, max, R, C, d_\Sigma)$ 
if  $b_\Sigma = 0$  :
    return 0
return 1

```

$Rverify(Q, min, max, R, C, \pi)$

```

 $sum \leftarrow 0$ 
 $i \leftarrow min$ 
 $(Astr, Bstr, cstr, rstr) \leftarrow \pi$ 
 $A_{min} \parallel \dots \parallel A_{max} \leftarrow Astr$ 
 $B_{min} \parallel \dots \parallel B_{max} \leftarrow Bstr$ 
 $c_{min} \parallel \dots \parallel c_{max} \leftarrow cstr$ 
 $r_{min} \parallel \dots \parallel r_{max} \leftarrow rstr$ 
while  $i \leq max$  :
    if  $(c_i = 0) \vee (r_i = 0)$  :
        return 0
    if  $r_i P \neq A_i + c_i R$  :
        return 0
    if  $r_i Q \neq B_i + c_i (C - iP)$  :
        return 0
     $sum \leftarrow sum + c_i \mod q$ 
     $i \leftarrow i + 1$ 
 $c \leftarrow H(P \parallel Q \parallel R \parallel C \parallel Astr \parallel Bstr)$ 
if  $c = sum \mod q$  :
    return 1
return 0

```

Гомоморфное сложение $SE.Add$

Вход:

- n шифртекстов $(c_1^1, \dots, c_t^1), \dots, (c_1^n, \dots, c_t^n)$.

Выход:

- шифртекст $(c_1, \dots, c_t)$, где $c_i = c_i^1 + \dots + c_i^n, i = 1, \dots, t$.

$Add((c_1^1, \dots, c_t^1), \dots, (c_1^n, \dots, c_t^n))$

```

for  $j = 1, \dots, n$  :
     $((R_1^j, C_1^j), \dots, (R_t^j, C_t^j)) \leftarrow (c_1^j, \dots, c_t^j)$ 
for  $i = 1, \dots, t$  :
     $R_i \leftarrow \sum_{j=1}^n R_i^j, C_i \leftarrow \sum_{j=1}^n C_i^j$ 
 $(c_1, \dots, c_t) \leftarrow (R_1, C_1), \dots, (R_t, C_t)$ 
return  $(c_1, \dots, c_t)$ 

```

Частичное расшифрование $SE.DecPart$

Вход:

- контекстно-зависимая строка ctx_{DP} ,
- частичный ключ расшифрования sk^{part} ,
- частичный ключ шифрования pk^{part} ,
- шифртекст $(c_1, \dots, c_t)$, $t \in \mathbb{N}$.

Выход:

- результат частичного расшифрования, $(\mu_1, \dots, \mu_t)$, $t \in \mathbb{N}$,
- доказательство π корректности частичного расшифрования (при расшифровании использовалось значение sk^{part} , соответствующее pk^{part}).

<u>$DecPart(ctx_{DP}, sk^{part}, pk^{part}, (c_1, \dots, c_t))$</u>	<u>$Dprove(ctx, x, Y_1, G_1, Y_2, G_2)$</u>
$(d, Q) \leftarrow (sk^{part}, pk^{part})$	$u \xleftarrow{\mathcal{U}} \mathbb{Z}_q^*$
$(R_1, C_1), \dots, (R_t, C_t) \leftarrow (c_1, \dots, c_t)$	$U_1 \leftarrow uG_1$
for $i = 1, \dots, t$:	$U_2 \leftarrow uG_2$
$\mu_i \leftarrow dR_i$	$v \leftarrow Hash_{DP}(ctx \ U_1 \ U_2 \ G_1 \ Y_1 \ G_2 \ Y_2)$
$\pi_i \leftarrow Dprove(ctx_{DP}, d, \mu_i, R_i, Q, P)$	$w \leftarrow xv + u \mod q$
$\pi \leftarrow (\pi_1, \dots, \pi_t)$	$\pi \leftarrow (w, U_1, U_2)$
return $((\mu_1, \dots, \mu_t), \pi)$	return π

Проверка корректности частичного расшифрования $SE.VerifyDecPart$

Вход:

- контекстно-зависимая строка ctx_{DP} ,
- частичный ключ шифрования pk^{part} ,
- шифртекст $(c_1, \dots, c_t)$, $t \in \mathbb{N}$,
- результат частичного расшифрования $(\mu_1, \dots, \mu_t)$,
- доказательство π корректности частичного расшифрования.

Выход:

- бит $b \in \{0,1\}$: $b = 1$ ($b = 0$) – проверка завершилась (не)успешно.

```

 $VerifyDecPart(ctx_{DP}, pk^{part}, (c_1, \dots, c_t), (\mu_1, \dots, \mu_t), \pi)$ 
 $Q \leftarrow pk^{part}$ 
 $(R_1, C_1), \dots, (R_t, C_t) \leftarrow (c_1, \dots, c_t)$ 
 $(\pi_1, \dots, \pi_t) \leftarrow \pi$ 
for  $i = 1, \dots, t$  :
     $b_i \leftarrow Dverify(ctx_{DP}, \mu_i, R_i, Q, P, \pi_i)$ 
    if  $b_i = 0$  :
        return 0
return 1

```

$$\begin{array}{l}
\text{Dverify}(ctx, Y_1, G_1, Y_2, G_2, \pi) \\
\hline
(w, U_1, U_2) \leftarrow \pi \\
v \leftarrow \text{Hash}_{DP}(ctx \| U_1 \| U_2 \| G_1 \| Y_1 \| G_2 \| Y_2) \\
\text{if } (wG_1 = vY_1 + U_1) \wedge (wG_2 = vY_2 + U_2): \\
\quad \text{return 1} \\
\text{return 0}
\end{array}$$

Агрегирование результатов расшифрования $SE.DecAgg$

Вход:

- шифртекст $(c_1, \dots, c_t)$, $t \in \mathbb{N}$,
- результат частичного расшифрования $(\mu_1^1, \dots, \mu_t^1)$,
- результат частичного расшифрования $(\mu_1^2, \dots, \mu_t^2)$,
- частичный ключ шифрования pk_1^{part} ,
- частичный ключ шифрования pk_2^{part} .

Выход:

- открытый текст $(m_1, \dots, m_t)$.

$$\begin{array}{l}
\text{DecAgg}((c_1, \dots, c_t), (\mu_1^1, \dots, \mu_t^1), (\mu_1^2, \dots, \mu_t^2), pk_1^{part}, pk_2^{part}) \\
\hline
(R_1, C_1), \dots, (R_t, C_t) \leftarrow (c_1, \dots, c_t) \\
(Q_1, Q_2) \leftarrow (pk_1^{part}, pk_2^{part}) \\
h_1 \leftarrow H(Q_2 \| Q_1) \\
h_2 \leftarrow H(Q_1 \| Q_2) \\
\text{for } i = 1, \dots, t: \\
\quad M_i \leftarrow C_i - h_1 \mu_i^1 - h_2 \mu_i^2 \\
\quad \text{find } m_i: M_i = m_i P \\
\text{return } (m_1, \dots, m_t)
\end{array}$$

2.5 Схема разделения секрета (с доверенным дилером)

Схема разделения секрета позволяет разделить секрет s между n участниками таким образом, что любые k участников, объединившись, могут восстановить секрет. Схема разделения секрета SS определяется следующими процессами (используется схема, определенная в работе [9]).

Разделение секрета $SS.Share$

Вход:

- секрет s ,
- общее число участников $n \in \mathbb{N}$,
- пороговое число участников $k \in \mathbb{N}$, $k \leq n$.

Выход:

- доли $(s_1, \dots, s_n)$.

Share(s, n, k)

```

for  $i = 1 \dots k - 1$  :
     $\alpha_i \xleftarrow{\mathcal{U}} \mathbb{Z}_q$ 
for  $i = 1 \dots n$  :
     $s_i \leftarrow s + \sum_{j=1}^{k-1} \alpha_j \cdot i^j \bmod q$ 
return ( $s_1, \dots, s_n$ )

```

Восстановление секрета *SS.Comb*

Вход:

- множество номеров $\{i_1, \dots, i_k\} \subseteq \{1, \dots, n\}$,
- доли $(s_{i_1}, \dots, s_{i_k})$.

Выход:

- секрет s .

Comb($\{i_1, \dots, i_k\}, (s_{i_1}, \dots, s_{i_k})$)

```

for  $i = i_1 \dots i_k$  :
     $l_i \leftarrow \prod_{j=\{i_1, \dots, i_k\} \setminus \{i\}} \frac{-j}{i-j} \bmod q$ 
 $s \leftarrow \sum_{i=\{i_1, \dots, i_k\}} l_i \cdot s_i \bmod q$ 
return  $s$ 

```

2.6 Схема коммитмента (обязательств)

Схема коммитмента C определяется следующими процессами.

Генерация ключа *C.KeyGen*

Выход:

- ключ K

KeyGen()

```

 $K \xleftarrow{\mathcal{U}} \{0, 1\}^{256}$ 
return  $K$ 

```

Формирование коммитмента *C.Cmt*

Вход:

- ключ K ,
- скрываемое значение v .

Выход:

- значение коммитмента com .

$$\frac{Cmt(K, v)}{com \leftarrow HMAC(K, v)}$$

return *com*

Проверка (раскрытие) коммитмента *C.Open*

Вход:

- ключ K ,
- скрываемое значение v ,
- значение коммитмента com .

Выход:

- бит $b \in \{0,1\}$: $b = 1$ ($b = 0$) – проверка завершилась (не)успешно.

$$\frac{Open(K, v, com)}{com' \leftarrow HMAC(K, v)}$$

if $com' = com$:

$b \leftarrow 1$

else :

$b \leftarrow 0$

return b

3 Участники протокола

В настоящем подразделе кратко перечислены основные участники протокола ДЭГ и их функции.

Избиратель – участник, являющийся гражданином, обладающим активным избирательным правом и включенным в списки избирателей.

Организатор – участник, осуществляющий организацию процесса голосования (в том числе соответствующая избирательная комиссия).

Аутентификатор – участник, осуществляющий идентификацию и аутентификацию избирателей (с помощью ЕСИА).

Регистратор – участник, предоставляющий избирателям право получения бюллетеня (путем формирования подписи вслепую).

Урна – участник, выдающий избирателям бюллетени и принимающий обратно заполненные зашифрованные бюллетени.

Учетчик – участник, представляющий собой хранилище транзакций (бюллетеней избирателей и других данных), к которому есть постоянный доступ на запись и/или чтение у участников Урна, Организатор, Регистратор, Наблюдатель, а также осуществляющий подсчет итогов.

Наблюдатель – участник, осуществляющий наблюдение за процессом голосования и аудит результатов голосования; имеет доступ к хранилищу транзакций, предоставляет публичный доступ ко всему содержимому данного хранилища через Портал наблюдения.

4 Этапы протокола

В настоящем подразделе описаны шаги выполнения протокола ДЭГ для n независимых процедур голосования.

Будем считать, что взаимодействие между всеми участниками (в том числе с внешними участниками) осуществляется защищенным образом (обеспечивается аутентификация участников и конфиденциальность передаваемых данных), сообщения от Организатора, Регистратора и Учетчика, публикующих данные в Учетчике, аутентифицированы. При описании протокола не указываются особенности работы системы, с помощью которой он может быть реализован (например, способы аутентификации участников и пр.).

4.1 Формирование исходных данных

4.1.1 Формирование списков избирателей

1) Избиратель с помощью Единого портала государственных услуг подает заявление на участие в ДЭГ (заявление включает идентификатор СНИЛС и номер паспорта *passport* Избирателя).

2) На основании заявлений избирателей Центральная избирательная комиссия (ЦИК) для каждого $i \in \{1, \dots, n\}$ формирует списки избирателей, участвующих в ДЭГ.

3) Организатор генерирует в системе ключ для схемы коммитмента $K = C.KeyGen()$.

4.1.2 Передача данных голосования

1) ЦИК передает Организатору данные голосования для каждого $i \in \{1, \dots, n\}$, в том числе списки избирателей.

4.2 Инициализация системы

4.2.1 Генерация ключей

1) Организатор генерирует частичный (разделяемый) ключ шифрования

$$(sk_{\text{дел}}, pk_{\text{дел}}) = SE.KeyGen()$$

на выделенном устройстве. Закрытый ключ $sk_{\text{дел}}$ разделяется по схеме разделения секрета с помощью функции $SS.Share(sk_{\text{дел}}, N, k)$ между N держателями долей ключа (с некоторым параметром $k \leq N$), вычисленные доли s_i записываются на защищенные носители, которые отдаются держателям долей ключа. Ключ $sk_{\text{дел}}$ уничтожается на устройстве.

2) Учетчик для каждого $i \in \{1, \dots, n\}$ генерирует частичный (неразделяемый) ключ шифрования:

$$(sk_{\text{недел}}^i, pk_{\text{недел}}^i) = SE.KeyGen().$$

3) Регистратор для каждого $i \in \{1, \dots, n\}$ генерирует ключ подписи вслепую:

$$(sk_{\text{blind}}^i, pk_{\text{blind}}^i) = BS.KeyGen().$$

4.2.2 Загрузка данных голосования

1) Организатор передает Регистратору список избирателей.

2) Организатор передает Учетчику данные голосования для каждого $i \in \{1, \dots, n\}$, содержащие:

- идентификатор процедуры голосования ($VotingID_i$);
- хэш-значение от текста бюллетеня;
- общее количество вариантов t_i , максимальное и минимальное количество выбранных вариантов min_i, max_i ($1 \leq min_i \leq max_i < t_i$).

3) Регистратор передает Учетчику ключи $pk_{blind}^i, i = 1, \dots, n$.

4) Учетчик для каждого $i \in \{1, \dots, n\}$ публикует данные голосования совместно с соответствующими ключами pk_{blind}^i .

5) Организатор для каждого $i \in \{1, \dots, n\}$ формирует список значений коммитментов $com_{СНИЛС} = C.Cmt(K, \text{СНИЛС})$ для избирателей из соответствующих списков и публикует список коммитментов в Учетчике.

6) Организатор передает Регистратору ключ K .

7) Организатор передает Урне текст бюллетеня, идентификатор $VotingID_i$ и ключ pk_{blind}^i проверки подписи вслепую для всех $i \in \{1, \dots, n\}$.

4.2.3 Загрузка ключа шифрования бюллетеней

1) Организатор передает Учетчику разделяемый ключ шифрования $pk_{дел}$.

2) Учетчик для каждого $i \in \{1, \dots, n\}$ формирует ключ шифрования бюллетеней:

$$pk_i = SE.KeyAgg(pk_{дел}, pk_{недел}^i).$$

3) Учетчик для каждого $i \in \{1, \dots, n\}$ публикует:

- разделяемый ключ шифрования $pk_{дел}$;
- неразделяемый ключ шифрования $pk_{недел}^i$;
- итоговый ключ шифрования pk_i .

4) Учетчик передает Регистратору ключи $pk_i, i = 1, \dots, n$.

5) Организатор через Портал наблюдения проверяет, что:

- был опубликован известный ему разделяемый ключ шифрования $pk_{дел}$;
- для каждого $i \in \{1, \dots, n\}$ итоговый ключ pk_i был правильно вычислен.

4.3 Предоставление права получения бюллетеня

4.3.1 Идентификация и аутентификация избирателя

1) Избиратель обращается к Аутентификатору, который выполняет идентификацию и аутентификацию Избирателя с помощью ЕСИА, в результате чего Аутентификатор получает токен доступа, подписанный ЕСИА и содержащий информацию об Аутентификаторе (так называемый идентификатор клиента ЕСИА).

2) Аутентификатор передает Избирателю токен доступа.

4.3.2 Получение подписи вслепую и ключа шифрования бюллетеней

1) Избиратель передает токен доступа Регистратору.

2) Регистратор проверяет корректность токена доступа, в том числе подпись ЕСИА и соответствие информации об Аутентификаторе.

3) Регистратор с помощью токена доступа получает из ЕСИА идентификатор СНИЛС и номер паспорта *passport* Избирателя.

4) Регистратор проверяет, что:

- Избиратель есть в соответствующих списках избирателей;
- Избиратель не получал бюллетень.

Если какая-то из проверок завершена неуспешно, Регистратор возвращает Избирателю ошибку.

5) Регистратор передает Избирателю ключи $pk_{blind}^i, i = 1, \dots, n$.

6) Избиратель генерирует ключи подписи $(sk_{изб}^i, pk_{изб}^i) = S.KeyGen(), i = 1, \dots, n$.

7) Избиратель и Регистратор для каждого $i \in \{1, \dots, n\}$ выполняют протокол формирования подписи вслепую $\langle BS.Sign(sk_{blind}^i, pk_{blind}^i), BS.User(pk_{blind}^i, pk_{изб}^i) \rangle$, в результате чего на стороне Избирателя формируется значение подписи σ_{blind}^i .

8) Регистратор передает Избирателю ключи шифрования бюллетеней $pk_i, i = 1, \dots, n$, и соответствующие им идентификаторы голосования $VotingID_i$.

9) Регистратор записывает в списке избирателей паспортные данные Избирателя, фиксируя тем самым факт выдачи подписи вслепую.

10) Регистратор формирует значение коммитмента $com_{pas} = C.Cmt(K, СНИЛС \parallel passport)$ и для каждого $i \in \{1, \dots, n\}$ публикует его в Учетчике.

4.4 Подача заполненного бюллетеня

4.4.1 Волеизъявление

1) Избиратель анонимно обращается к Урне и запрашивает текст бюллетеня по идентификатору $VotingID_i$ для каждого $i \in \{1, \dots, n\}$.

2) Урна отправляет Избирателю текст бюллетеня для каждого $i \in \{1, \dots, n\}$.

3) Избиратель для каждого $i \in \{1, \dots, n\}$ заполняет бюллетень, определяя набор $(m_1^i, \dots, m_{t_i}^i)$, где значение $m_j^i \in \{0, 1\}$ обозначает голос «за» (1) или «против» (0) за j -й вариант, $j = 1, \dots, t_i$, и зашифровывает его:

$$((c_1^i, \dots, c_{t_i}^i), d^i) = SE.Enc(pk_i, (m_1^i, \dots, m_{t_i}^i), min_i, max_i).$$

4) Избиратель для каждого $i \in \{1, \dots, n\}$ формирует транзакцию tx^i , включающую следующие данные:

- идентификатор $VotingID_i$;
- зашифрованный бюллетень $((c_1^i, \dots, c_{t_i}^i), d^i)$;
- открытый ключ Избирателя $pk_{изб}^i$;
- значение подписи σ_{blind}^i .

5) Избиратель для каждого $i \in \{1, \dots, n\}$ подписывает транзакцию tx^i :

$$\sigma^i = S.Sign(sk_{изб}^i, tx^i).$$

6) Избиратель отправляет Урне подписанные транзакции $(tx^i, \sigma^i), i = 1, \dots, n$.

7) Урна проверяет для каждого $i \in \{1, \dots, n\}$:

- корректность формата транзакции (наличие необходимых полей, их длина, и т.д.);

- корректность подписи Регистратора в транзакции (с помощью функции $BS.Verify(pk_{blind}^i, pk_{изб}^i, \sigma_{blind}^i)$);
- корректность подписи Избирателя под транзакцией (с помощью функции $S.Verify(pk_{изб}^i, tx^i, \sigma^i)$);
- уникальность ключа $pk_{изб}^i$ (по своей внутренней базе данных).

Если какая-то из проверок завершена неуспешно, Урна возвращает Избирателю ошибку и не принимает бюллетень.

В противном случае Урна:

- добавляет ключ $pk_{изб}^i$ в свою внутреннюю базу данных;
- отправляет транзакции Учетчику (не подписывая дополнительно);
- формирует квитанцию о приеме транзакции (подписанная Урной принятая транзакция и ее идентификатор) и отображает ее Избирателю.

На экране устройства избирателя отображается содержимое его транзакции и квитанции, которые избиратель может скопировать для дальнейшей проверки корректности учета.

4.4.2 Публикация бюллетеня

Учетчик проверяет для каждого $i \in \{1, \dots, n\}$:

- корректность формата транзакции (наличие необходимых полей, их длина, и т.д.);
- корректность подписи Регистратора в транзакции (с помощью функции $BS.Verify(pk_{blind}^i, pk_{изб}^i, \sigma_{blind}^i)$);
- корректность подписи Избирателя под транзакцией (с помощью функции $S.Verify(pk_{изб}^i, tx^i, \sigma^i)$);
- факт однократности транзакции с таким ключом Избирателя.

Если проверки пройдены, транзакции добавляются в Учетчик.

4.4.3 Проверка бюллетеня

1) Избиратель с помощью Портала наблюдения проверяет (поиском по идентификатору транзакции из квитанции от Урны), что его транзакции находятся в хранилище. Если Избиратель не находит свои транзакции, то он обращается к Наблюдателю с жалобой, направляя ему свои транзакции и квитанции.

2) Если Наблюдатель получил жалобу, то он проверяет корректность транзакций (подписи Регистратора и Избирателя), квитанций (подпись Урны) и отсутствие в хранилище других транзакций с данным ключом. Если проверка завершилась успешно, Наблюдатель фиксирует факт некорректной работы системы.

4.5 Подведение итогов

4.5.1 Прекращение выдачи и приема бюллетеней

- 1) По завершении голосования Регистратор прекращает выдачу подписей вслепую.
- 2) По истечению периода времени, предоставленному Избирателю для завершения голосования (в соответствии с параметром, установленным оператором, например – 15 минут), Организатор сообщает Урне и Учетчику о завершении приема бюллетеней.

3) Учетчик публикует транзакцию о завершении приема бюллетеней. После этого Учетчик не принимает бюллетени.

4.5.2 Восстановление разделяемого ключа шифрования

- 1) Организатор проводит церемонию сборки разделяемого ключа расшифрования $sk_{\text{дел}}$ с помощью функции $SS.Comb(\{i_1, \dots, i_k\}, \{s_{i_1}, \dots, s_{i_k}\})$. Для сборки ключа достаточно k долей.
- 2) Организатор передает ключ $sk_{\text{дел}}$ Учетчику.

4.5.3 Подсчет итогов

- 1) Учетчик осуществляет проверку корректности шифрования для хранящихся бюллетеней с помощью функции:

$$SE.VerifyEnc(pk_i, (c_1^i, \dots, c_{t_i}^i), d^i, min_i, max_i).$$

- 2) Учетчик для каждого $i \in \{1, \dots, n\}$ осуществляет гомоморфное сложение шифртекстов $(c_1^i, \dots, c_{t_i}^i)$, соответствующих успешно проверенным бюллетеням, с помощью функции $SE.Add$, формируя тем самым итоговый шифртекст $(\bar{c}_1^i, \dots, \bar{c}_{t_i}^i)$.

4.5.4 Расшифрование итогов

- 1) Учетчик для каждого $i \in \{1, \dots, n\}$ осуществляет частичное расшифрование значения $(\bar{c}_1^i, \dots, \bar{c}_{t_i}^i)$, используя части ключей:

- вычисляет:

$$(\mu_{\text{недел}}^i, \pi_{\text{недел}}^i) = SE.DecPart(VotingID_i, sk_{\text{недел}}^i, pk_{\text{недел}}^i, (\bar{c}_1^i, \dots, \bar{c}_{t_i}^i));$$

- проверяет значение $(\mu_{\text{недел}}^i, \pi_{\text{недел}}^i)$ с помощью функции:

$$SE.VerifyDecPart(VotingID_i, pk_{\text{недел}}^i, (\bar{c}_1^i, \dots, \bar{c}_{t_i}^i), \mu_{\text{недел}}^i, \pi_{\text{недел}}^i);$$

- вычисляет:

$$(\mu_{\text{дел}}^i, \pi_{\text{дел}}^i) = SE.DecPart(VotingID_i, sk_{\text{дел}}^i, pk_{\text{дел}}^i, (\bar{c}_1^i, \dots, \bar{c}_{t_i}^i));$$

- проверяет значение $(\mu_{\text{дел}}^i, \pi_{\text{дел}}^i)$ с помощью функции:

$$SE.VerifyDecPart(VotingID_i, pk_{\text{дел}}^i, (\bar{c}_1^i, \dots, \bar{c}_{t_i}^i), \mu_{\text{дел}}^i, \pi_{\text{дел}}^i).$$

- 2) Учетчик для каждого $i \in \{1, \dots, n\}$ публикует значения $(\mu_{\text{недел}}^i, \pi_{\text{недел}}^i), (\mu_{\text{дел}}^i, \pi_{\text{дел}}^i)$.

- 3) Учетчик для каждого $i \in \{1, \dots, n\}$ осуществляет агрегирование результатов расшифрования, формируя результаты голосования:

$$result_i = SE.DecAgg((\bar{c}_1^i, \dots, \bar{c}_{t_i}^i), \mu_{\text{недел}}^i, \mu_{\text{дел}}^i, pk_{\text{недел}}^i, pk_{\text{дел}}^i).$$

- 4) Учетчик для каждого $i \in \{1, \dots, n\}$ публикует значения $result_i$.

4.6 Аудит

Наблюдатель проверяет:

- количество бюллетеней, которые оказались в Учетчике, не превышает количества Избирателей, которым выдали подпись вслепую (с учетом n процедур голосования);

- корректность подписи Регистратора на всех бюллетенях (с помощью функции *BS.Verify*);
- корректность подписи Избирателя на всех бюллетенях (с помощью функции *S.Verify*);
- уникальность транзакции с указанным ключом проверки подписи;
- доказательства корректности содержимого бюллетеней (с помощью функции *SE.VerifyEnc*).
- осуществляет гомоморфное сложение успешно проверенных бюллетеней (с помощью функции *SE.Add*);
- проверяет доказательства корректности частичного расшифрования (с помощью функции *SE.VerifyDecPart*);
- осуществляет агрегирование результатов расшифрования (с помощью функции *SE.DecAgg*) и сравнивает полученное значение с опубликованным.

4.7 Публикация результатов

В случае успешного аудита Наблюдатель информирует Организатора о своем согласии с итогами. После этого Организатор для каждого $i \in \{1, \dots, n\}$ получает из Учетчика значение $result_i$ и формирует итоги голосования.

5 Совместимость с предыдущими версиями протокола

5.1 Возможность изменения протокола подписи вслепую

Протокол тайного голосования предусматривает изменение протокола использования подписи вслепую с протокола TeZhu 2022 на протокол на основе RSA установкой соответствующих системных параметров.

В этом случае протокол формирования и проверки подписи вслепую задается следующими алгоритмами:

- $KeyGen(k) \rightarrow (sk, pk)$: алгоритм генерации ключей, принимающий на вход параметр безопасности k и возвращающий пару ключей (sk, pk) , где sk – ключ подписи, pk – ключ проверки подписи;
- $\langle Sign(sk), User(pk, m) \rangle \rightarrow (b, \sigma)$: интерактивный протокол, выполняемый между подписывающим, который обладает ключом подписи sk , и клиентом, который обладает сообщением m ; подписывающий выдает $b = 1$, если взаимодействие успешно завершилось, и $b = 0$ в противном случае; клиент выдает значение подписи σ в случае успешного завершения протокола и код ошибки в противном случае;
- $Verify(pk, m, \sigma) \rightarrow b$: детерминированный алгоритм проверки подписи, принимающий на вход ключ проверки подписи pk , сообщение m и подпись σ и возвращающий единицу, если значение подписи верное, и ноль в противном случае.

В качестве протокола формирования и проверки подписи вслепую используется схема подписи вслепую на основе RSA, предложенная в работе [5] (см. п. 5.1.1), при этом для хэширования сообщения используется конструкция FDH, определенная в п. 5.1.2.

5.1.1 Схема подписи вслепую на основе RSA

Параметр безопасности k выбирается равным 4096.

Алгоритм генерации ключей¹:

KeyGen(k)

$p, q \leftarrow \mathbb{Z}_{2^{k/2}} :$
 $p, q - \text{prime}$
 $\sqrt{2} \cdot 2^{k/2-1} \leq p \leq 2^{k/2} - 1$
 $\sqrt{2} \cdot 2^{k/2-1} \leq q \leq 2^{k/2} - 1$
 $N \leftarrow pq$
 $e \leftarrow 2^{16} + 1$
 $d \leftarrow e^{-1} \bmod \text{LCM}(p-1, q-1)$
return $(d, (e, N))$

Интерактивный протокол формирования подписи:

Sign (d)	User ($(e, N), m$)
	$h \leftarrow \text{FDH}(m, N)$
	$r \leftarrow \mathbb{Z}_N^*$
	$h' \leftarrow r^e h$
	$\xleftarrow{h'}$
$s \leftarrow (h')^d$	
	$\xrightarrow{s}$
return 1	$\sigma \leftarrow r^{-1}s$

Алгоритм проверки подписи:

Verify($(e, N), m, \sigma$)

$h \leftarrow \text{FDH}(m, N)$
if $\sigma^e = h$: **return** 1
else : **return** 0

5.1.2 Описание конструкции FDH

В качестве базовой хэш-функции H для конструкции FDH используется хэш-функция «Стрибог» с длиной выхода 256 битов, определенная в ГОСТ Р 34.11-2012 [1].

Пусть $flag_1, flag_2$ – фиксированные различные строки длины 1 байт.

¹ $(sk, pk) \leftarrow (d, (e, N))$, см. 5.1

Конструкция FDH:

$FDH(m, N)$

$IV \leftarrow 0$

$first_block \leftarrow \text{int}(\text{msb}_{256}(\text{str}_{4096}(N)))$

while $\text{int}(H(m\|N\|flag_1\|IV)) \geq first_block :$

$IV++$

$fdh \leftarrow H(m\|N\|flag_1\|IV) \parallel H(m\|N\|flag_2\|IV+1) \parallel \dots \parallel H(m\|N\|flag_2\|IV+15)$

return fdh

При хэшировании значения IV и N представляются как байтовые строки длиной 1 байт и 512 байт соответственно (в формате big-endian).

Список литературы

- [1] ГОСТ Р 34.10–2012. Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи. М.: Стандартинформ, 2013.
- [2] ГОСТ Р 34.11–2012. Информационная технология. Криптографическая защита информации. Функция хэширования. М.: Стандартинформ, 2013.
- [3] Р 50.1.113–2016. Информационная технология. Криптографическая защита информации. Криптографические алгоритмы, сопутствующие применению алгоритмов электронной цифровой подписи и функции хэширования. М.: Стандартинформ, 2016.
- [4] Р 1323565.1.024–2019. Информационная технология. Криптографическая защита информации. Параметры эллиптических кривых для криптографических алгоритмов и протоколов. М.: Стандартинформ, 2019.
- [5] Bellare M. et al. The One-More-RSA-Inversion Problems and the Security of Chaum's Blind Signature Scheme // Journal of Cryptology. – 2003. – Vol. 16. – No. 3. – P. 185–215.
- [6] ElGamal T. A public key cryptosystem and a signature scheme based on discrete logarithms // IEEE Transactions on Information Theory. 1985. Vol. 31. No. 4. P. 469–472.
- [7] Chaum D., Pedersen T. P. Wallet Databases with Observers // Advances in Cryptology – CRYPTO'92. Lecture Notes in Computer Science. 1993. Vol. 740. P. 89–105.
- [8] Cramer R., Damgård I., Schoenmakers B. Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols // Advances in Cryptology – CRYPTO'94. Lecture Notes in Computer Science. 1994. Vol. 839. P. 174–187.
- [9] Shamir A. How to Share a Secret // Communications of the ACM. 1979. Vol. 22. No. 11. P. 612–613.
- [10] Tessaro S., Zhu C. Short Pairing-Free Blind Signatures with Exponential Security // Advances in Cryptology – EUROCRYPT 2022. Lecture Notes in Computer Science. 2022. Vol. 13276. P. 782–811.